Python

Introduction to Python

Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability.

Key Features

- Easy to learn and read
- Interpreted language (no compilation needed)
- Dynamically typed
- Cross-platform
- Extensive standard library
- Strong community support

Python Use Cases

- Web Development (Django, Flask)
- Data Science and Machine Learning
- Automation and Scripting
- Scientific Computing
- Game Development
- Desktop Applications

Python Versions

- Python 2.x (deprecated, no longer maintained)
- Python 3.x (current, recommended)

Installation and Setup

Installing Python

Windows:

```
# Download from python.org
# Or use Windows Store
# Or use chocolatey
choco install python
```

macOS:

```
# Using Homebrew
brew install python3
```

Linux:

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install python3 python3-pip

# Fedora
sudo dnf install python3 python3-pip
```

Verify Installation

```
python --version
# or
python3 --version

# Check pip
pip --version
# or
pip3 --version
```

Running Python

Interactive Mode (REPL):

```
python
>>> print("Hello, World!")
Hello, World!
>>> exit()
```

Script Mode:

```
# Create file: hello.py
python hello.py
```

Python IDEs and Editors

- PyCharm
- VS Code
- Jupyter Notebook
- Sublime Text
- IDLE (comes with Python)

Basic Syntax

Hello World

```
print("Hello, World!")
```

Comments

```
# This is a single-line comment

"""
This is a multi-line comment
or docstring
"""

'''
This is also a multi-line comment
'''
```

Indentation

```
# Python uses indentation to define code blocks
if True:
    print("This is indented")
    print("Same level of indentation")
print("This is not indented")
```

Multiple Statements

```
# Multiple statements on one line
x = 5; y = 10; print(x + y)

# Line continuation
total = 1 + 2 + 3 + \
        4 + 5 + 6

# Implicit line continuation
total = (1 + 2 + 3 +
         4 + 5 + 6)
```

Case Sensitivity

```
# Python is case-sensitive
name = "John"
Name = "Jane"
print(name)  # John
print(Name)  # Jane
```

Variables and Data Types

Variables

```
# No declaration needed
x = 5
name = "Alice"
is_active = True

# Multiple assignment
a, b, c = 1, 2, 3
x = y = z = 0

# Swapping variables
a, b = 10, 20
a, b = b, a  # Now a=20, b=10
```

Variable Naming Rules

```
# Valid variable names
my_variable = 1
myVariable = 2
_my_variable = 3
myVariable2 = 4

# Invalid variable names (will cause errors)
# 2myVariable = 5  # Can't start with number
# my-variable = 6  # Can't use hyphens
# my variable = 7  # Can't use spaces
```

Numeric Types

Integer

```
x = 10
y = -5
big_number = 1_000_000  # Underscores for readability
```

```
# Integer operations
print(10 + 5)   # 15
print(10 - 5)   # 5
print(10 * 5)   # 50
print(10 / 5)   # 2.0 (always returns float)
print(10 // 5)  # 2 (floor division)
print(10 % 3)   # 1 (modulus)
print(10 ** 2)  # 100 (exponentiation)
```

Float

```
x = 3.14
y = -2.5
scientific = 3.14e2  # 314.0

# Float precision
print(0.1 + 0.2)  # 0.30000000000000004
```

Complex Numbers

```
z = 3 + 4j
print(z.real)  # 3.0
print(z.imag)  # 4.0
```

Boolean

```
is_active = True
is_deleted = False

# Boolean operations
print(True and False)  # False
print(True or False)   # True
print(not True)        # False

# Truthy and Falsy values
# Falsy: False, None, 0, 0.0, "", [], {}, ()
# Everything else is Truthy
if []:
    print("This won't print")
if [1, 2, 3]:
    print("This will print")
```

String

```python
# Single or double quotes
name = 'Alice'
message = "Hello, World!"

# Multi-line strings
text = """This is a
multi-line
string"""

# String concatenation
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name

# String repetition
print("Ha" * 3)  # HaHaHa
```

None Type

```python
x = None
print(x)  # None
print(type(x))  # <class 'NoneType'>

# Checking for None
if x is None:
    print("x is None")
```

Type Checking and Conversion

```python
# Type checking
x = 5
print(type(x))  # <class 'int'>

# Type conversion
x = 5
y = float(x)     # 5.0
z = str(x)       # "5"
a = int("10")    # 10
b = bool(1)      # True

# Check instance
print(isinstance(5, int))  # True
print(isinstance(5.0, int))  # False
```

Operators

Arithmetic Operators

```
a = 10
b = 3

print(a + b)   # 13 (Addition)
print(a - b)   # 7 (Subtraction)
print(a * b)   # 30 (Multiplication)
print(a / b)   # 3.333... (Division)
print(a // b)  # 3 (Floor division)
print(a % b)   # 1 (Modulus)
print(a ** b)  # 1000 (Exponentiation)
```

Comparison Operators

```
a = 10
b = 5

print(a == b)  # False (Equal to)
print(a != b)  # True (Not equal to)
print(a > b)   # True (Greater than)
print(a < b)   # False (Less than)
print(a >= b)  # True (Greater than or equal to)
print(a <= b)  # False (Less than or equal to)
```

Logical Operators

```
x = True
y = False

print(x and y)  # False
print(x or y)   # True
print(not x)    # False

# Short-circuit evaluation
result = True or expensive_function()  # expensive_function() not called
```

Assignment Operators

```
x = 5
x += 3   # x = x + 3 (8)
x -= 2   # x = x - 2 (6)
x *= 4   # x = x * 4 (24)
x /= 3   # x = x / 3 (8.0)
x //= 2  # x = x // 2 (4.0)
```

```
x %= 3   # x = x % 3 (1.0)
x **= 2  # x = x ** 2 (1.0)
```

Identity Operators

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is c)      # True (same object)
print(a is b)      # False (different objects)
print(a == b)      # True (same value)
print(a is not b)  # True
```

Membership Operators

```
numbers = [1, 2, 3, 4, 5]

print(3 in numbers)      # True
print(10 in numbers)     # False
print(10 not in numbers) # True

# Works with strings too
print('a' in 'apple')    # True
```

Bitwise Operators

```
a = 5  # 0101 in binary
b = 3  # 0011 in binary

print(a & b)   # 1 (AND)
print(a | b)   # 7 (OR)
print(a ^ b)   # 6 (XOR)
print(~a)      # -6 (NOT)
print(a << 1)  # 10 (Left shift)
print(a >> 1)  # 2 (Right shift)
```

Operator Precedence

```
# Parentheses > Exponentiation > Multiplication/Division >
Addition/Subtraction
result = 2 + 3 * 4      # 14
result = (2 + 3) * 4    # 20
```

```
result = 2 ** 3 ** 2    # 512 (right to left)
result = (2 ** 3) ** 2  # 64
```

Control Flow

If Statement

```
# Simple if
x = 10
if x > 5:
    print("x is greater than 5")

# if-else
age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")

# if-elif-else
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(f"Grade: {grade}")

# Nested if
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
```

Ternary Operator

```
# Conditional expression
age = 20
status = "Adult" if age >= 18 else "Minor"

# Nested ternary
```

```
x = 10
result = "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
```

While Loop

```
# Basic while loop
count = 0
while count < 5:
    print(count)
    count += 1

# While with else
count = 0
while count < 5:
    print(count)
    count += 1
else:
    print("Loop completed")

# Infinite loop (use Ctrl+C to stop)
# while True:
#     print("Infinite loop")
```

For Loop

```
# Iterate over sequence
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Iterate over string
for char in "Python":
    print(char)

# Range function
for i in range(5):  # 0 to 4
    print(i)

for i in range(2, 10):  # 2 to 9
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
    print(i)

# For with else
for i in range(5):
    print(i)
else:
    print("Loop completed")
```

```python
# Enumerate
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Start enumerate from 1
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
```

Break and Continue

```python
# Break statement
for i in range(10):
    if i == 5:
        break
    print(i)  # Prints 0 to 4

# Continue statement
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)  # Prints odd numbers

# Break with else
for i in range(10):
    if i == 5:
        break
    print(i)
else:
    print("This won't print if break is executed")
```

Pass Statement

```python
# Placeholder for future code
for i in range(5):
    pass  # Do nothing

# Empty function
def my_function():
    pass

# Empty class
class MyClass:
    pass
```

Match Statement (Python 3.10+)

```python
# Pattern matching
def http_status(status):
    match status:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case _:
            return "Unknown"

# Match with conditions
def check_number(num):
    match num:
        case n if n < 0:
            return "Negative"
        case 0:
            return "Zero"
        case n if n > 0:
            return "Positive"
```

Functions

Basic Function

```python
# Function definition
def greet():
    print("Hello, World!")

# Function call
greet()
```

Function with Parameters

```python
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")

# Multiple parameters
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # 8
```

Default Parameters

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")              # Hello, Alice!
greet("Bob", "Hi")          # Hi, Bob!
greet("Charlie", greeting="Hey")  # Hey, Charlie!
```

Keyword Arguments

```
def introduce(name, age, city):
    print(f"I'm {name}, {age} years old, from {city}")

# Positional arguments
introduce("Alice", 25, "New York")

# Keyword arguments
introduce(name="Bob", city="London", age=30)

# Mixed (positional must come first)
introduce("Charlie", age=35, city="Paris")
```

Variable-Length Arguments

*args (Non-keyword variable arguments)

```
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_all(1, 2, 3))        # 6
print(sum_all(1, 2, 3, 4, 5))  # 15
```

**kwargs (Keyword variable arguments)

```
def print_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="New York")
```

Combining *args and **kwargs

```
def function_with_all(a, b, *args, **kwargs):
    print(f"a: {a}")
    print(f"b: {b}")
    print(f"args: {args}")
    print(f"kwargs: {kwargs}")

function_with_all(1, 2, 3, 4, 5, name="Alice", age=25)
```

Return Statement

```
# Single return value
def square(x):
    return x ** 2

# Multiple return values
def get_min_max(numbers):
    return min(numbers), max(numbers)

minimum, maximum = get_min_max([1, 2, 3, 4, 5])

# Early return
def is_even(num):
    if num % 2 == 0:
        return True
    return False
```

Docstrings

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle
        width (float): The width of the rectangle

    Returns:
        float: The area of the rectangle
    """
    return length * width

# Access docstring
print(calculate_area.__doc__)
```

Scope and Global Variables

```
# Global variable
global_var = 10

def function1():
    # Local variable
    local_var = 5
    print(global_var)  # Can access global
    print(local_var)

def function2():
    global global_var
    global_var = 20  # Modify global variable

# Nonlocal keyword (for nested functions)
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    inner()
    print(x)  # 20

outer()
```

Recursive Functions

```
# Factorial
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120

# Fibonacci
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(6))  # 8
```

Type Hints (Python 3.5+)

```
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

```python
def add(a: int, b: int) -> int:
    return a + b

# Optional type
from typing import Optional
def find_item(items: list, target: str) -> Optional[int]:
    if target in items:
        return items.index(target)
    return None
```

Data Structures

Lists

```python
# Create list
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
empty_list = []

# Access elements
print(fruits[0])   # apple (first element)
print(fruits[-1])  # cherry (last element)
print(fruits[1:3]) # ['banana', 'cherry'] (slicing)

# Modify list
fruits[0] = "apricot"
fruits.append("date")           # Add to end
fruits.insert(1, "blueberry")   # Insert at index
fruits.extend(["elderberry", "fig"])  # Add multiple items
fruits.remove("banana")         # Remove specific item
popped = fruits.pop()           # Remove and return last item
popped = fruits.pop(0)          # Remove and return item at index
del fruits[0]                   # Delete item at index
fruits.clear()                  # Remove all items

# List operations
numbers = [1, 2, 3, 4, 5]
print(len(numbers))      # 5 (length)
print(3 in numbers)      # True (membership)
print(max(numbers))      # 5
print(min(numbers))      # 1
print(sum(numbers))      # 15
numbers.sort()           # Sort in place
numbers.reverse()        # Reverse in place
sorted_nums = sorted(numbers)  # Return sorted copy
reversed_nums = list(reversed(numbers))  # Return reversed copy

# List methods
```

```python
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
print(numbers.count(1))       # 2 (count occurrences)
print(numbers.index(4))       # 2 (find index)
numbers.sort()                # [1, 1, 2, 3, 4, 5, 6, 9]
numbers.reverse()             # [9, 6, 5, 4, 3, 2, 1, 1]

# Copying lists
original = [1, 2, 3]
shallow_copy = original.copy()
shallow_copy2 = original[:]
shallow_copy3 = list(original)

# Nested lists
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[0][1])  # 2
```

Tuples

```python
# Create tuple
coordinates = (10, 20)
person = ("Alice", 25, "New York")
single_item = (42,)  # Note the comma
empty_tuple = ()

# Access elements
print(coordinates[0])   # 10
print(person[1:])       # (25, 'New York')

# Tuples are immutable
# coordinates[0] = 15  # Error!

# Tuple unpacking
x, y = coordinates
name, age, city = person

# Tuple methods
numbers = (1, 2, 3, 2, 4, 2)
print(numbers.count(2))  # 3
print(numbers.index(3))  # 2

# Named tuples
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y)  # 10 20
```

Sets

```
# Create set
fruits = {"apple", "banana", "cherry"}
numbers = {1, 2, 3, 4, 5}
empty_set = set()  # Note: {} creates empty dict

# Add/remove elements
fruits.add("date")
fruits.remove("banana")      # Error if not found
fruits.discard("banana")     # No error if not found
popped = fruits.pop()        # Remove and return arbitrary item
fruits.clear()               # Remove all items

# Set operations
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print(set1 | set2)           # {1, 2, 3, 4, 5, 6} (union)
print(set1 & set2)           # {3, 4} (intersection)
print(set1 - set2)           # {1, 2} (difference)
print(set1 ^ set2)           # {1, 2, 5, 6} (symmetric difference)

# Set methods
print(set1.union(set2))
print(set1.intersection(set2))
print(set1.difference(set2))
print(set1.symmetric_difference(set2))
print(set1.issubset(set2))
print(set1.issuperset(set2))

# Frozen set (immutable)
frozen = frozenset([1, 2, 3])
```

Dictionaries

```
# Create dictionary
person = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
}
empty_dict = {}

# Access elements
print(person["name"])         # Alice
print(person.get("age"))      # 25
print(person.get("country", "USA"))  # USA (default value)

# Modify dictionary
```

```python
person["age"] = 26            # Update value
person["country"] = "USA"     # Add new key-value pair
person.update({"job": "Engineer", "age": 27})

# Remove elements
del person["city"]
popped = person.pop("age")
popped_item = person.popitem()  # Remove and return last item
person.clear()                   # Remove all items

# Dictionary methods
person = {"name": "Alice", "age": 25, "city": "New York"}
print(person.keys())          # dict_keys(['name', 'age', 'city'])
print(person.values())        # dict_values(['Alice', 25, 'New York'])
print(person.items())         # dict_items([('name', 'Alice'), ...])

# Iterate over dictionary
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")

# Dictionary comprehension
squares = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Nested dictionaries
students = {
    "student1": {"name": "Alice", "grade": 90},
    "student2": {"name": "Bob", "grade": 85}
}

# Default dict
from collections import defaultdict
dd = defaultdict(int)  # Default value is 0
dd['count'] += 1

# Counter
from collections import Counter
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
counter = Counter(words)
print(counter)  # Counter({'apple': 3, 'banana': 2, 'cherry': 1})
print(counter.most_common(2))  # [('apple', 3), ('banana', 2)]
```

String Manipulation

String Basics

```python
# String creation
single = 'Hello'
double = "World"
multi = """This is a
multi-line string"""

# String concatenation
greeting = "Hello" + " " + "World"
greeting = "Hello" " " "World"  # Automatic concatenation

# String repetition
print("Ha" * 3)  # HaHaHa

# String length
print(len("Hello"))  # 5
```

String Indexing and Slicing

```python
text = "Python Programming"

# Indexing
print(text[0])    # P
print(text[-1])   # g
print(text[-2])   # n

# Slicing
print(text[0:6])   # Python
print(text[:6])    # Python (from start)
print(text[7:])    # Programming (to end)
print(text[::2])   # Pto rgamn (every 2nd char)
print(text[::-1])  # gnimmargorP nohtyP (reverse)
```

String Methods

```python
text = "  Hello World  "

# Case conversion
print(text.upper())       # "  HELLO WORLD  "
print(text.lower())       # "  hello world  "
print(text.capitalize())  # "  hello world  "
print(text.title())       # "  Hello World  "
print(text.swapcase())    # "  hELLO wORLD  "

# Whitespace removal
print(text.strip())       # "Hello World"
print(text.lstrip())      # "Hello World  "
print(text.rstrip())      # "  Hello World"
```

```
# Search and replace
text = "Hello World"
print(text.find("World"))     # 6
print(text.find("Python"))    # -1 (not found)
print(text.index("World"))    # 6 (raises error if not found)
print(text.replace("World", "Python"))  # "Hello Python"
print(text.count("l"))        # 3

# Check string properties
print("hello".isalpha())      # True
print("hello123".isalnum())   # True
print("123".isdigit())        # True
print("   ".isspace())        # True
print("Hello".islower())      # False
print("HELLO".isupper())      # True
print("Hello World".istitle()) # True

# Split and join
text = "apple,banana,cherry"
fruits = text.split(",")      # ['apple', 'banana', 'cherry']
joined = "-".join(fruits)     # "apple-banana-cherry"

# Starts with / ends with
print("Hello".startswith("He"))  # True
print("Hello".endswith("lo"))    # True

# Padding and alignment
print("Hello".center(10))     # "  Hello   "
print("Hello".ljust(10))      # "Hello     "
print("Hello".rjust(10))      # "     Hello"
print("42".zfill(5))          # "00042"
```

String Formatting

Old Style (%)

```
name = "Alice"
age = 25
print("My name is %s and I'm %d years old" % (name, age))
```

str.format()

```
name = "Alice"
age = 25
print("My name is {} and I'm {} years old".format(name, age))
print("My name is {0} and I'm {1} years old".format(name, age))
print("My name is {n} and I'm {a} years old".format(n=name, a=age))
```

```
# Number formatting
print("Pi is approximately {:.2f}".format(3.14159))  # 3.14
print("Number: {:05d}".format(42))  # 00042
```

f-strings (Python 3.6+)

```
name = "Alice"
age = 25
print(f"My name is {name} and I'm {age} years old")

# Expressions in f-strings
print(f"Next year I'll be {age + 1}")

# Formatting
pi = 3.14159
print(f"Pi is approximately {pi:.2f}")

# Dictionary in f-string
person = {"name": "Bob", "age": 30}
print(f"Name: {person['name']}, Age: {person['age']}")
```

String Operations

```
# Membership
print("Hello" in "Hello World")  # True
print("Python" in "Hello World") # False

# String multiplication
print("=" * 20)  # ====================

# Raw strings (ignore escape sequences)
path = r"C:\Users\name\Documents"
print(path)  # C:\Users\name\Documents

# Escape sequences
print("Line 1\nLine 2")  # New line
print("Tab\tseparated")  # Tab
print("He said \"Hello\"")  # Quotes
print("C:\\Users\\name")  # Backslash
```

File I/O

Reading Files

Read entire file

```python
# Method 1: Manual close
file = open('example.txt', 'r')
content = file.read()
print(content)
file.close()

# Method 2: Context manager (recommended)
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)
# File automatically closed after with block

# Read line by line
with open('example.txt', 'r') as file:
    for line in file:
        print(line.strip())

# Read all lines into list
with open('example.txt', 'r') as file:
    lines = file.readlines()
    print(lines)

# Read one line at a time
with open('example.txt', 'r') as file:
    line1 = file.readline()
    line2 = file.readline()
```

Writing Files

```python
# Write mode (overwrites existing file)
with open('output.txt', 'w') as file:
    file.write('Hello, World!\n')
    file.write('This is a new file.\n')

# Write multiple lines
lines = ['Line 1\n', 'Line 2\n', 'Line 3\n']
with open('output.txt', 'w') as file:
    file.writelines(lines)

# Append mode
with open('output.txt', 'a') as file:
    file.write('This line is appended.\n')
```

File Modes

```python
# 'r'  - Read (default)
# 'w'  - Write (overwrites)
# 'a'  - Append
```

```
# 'x'  - Exclusive creation (fails if file exists)
# 'b'  - Binary mode
# 't'  - Text mode (default)
# '+'  - Read and write

# Examples
with open('file.txt', 'r') as f:   # Read text
    pass

with open('file.bin', 'rb') as f:  # Read binary
    pass

with open('file.txt', 'w+') as f:  # Read and write
    pass
```

Working with File Paths

```
import os

# Check if file exists
if os.path.exists('file.txt'):
    print("File exists")

# Get file size
size = os.path.getsize('file.txt')

# Get absolute path
abs_path = os.path.abspath('file.txt')

# Join paths
path = os.path.join('folder', 'subfolder', 'file.txt')

# Directory operations
os.mkdir('new_folder')                # Create directory
os.makedirs('folder/subfolder')       # Create nested directories
os.listdir('.')                       # List files in directory
os.remove('file.txt')                 # Delete file
os.rmdir('folder')                    # Delete empty directory
os.rename('old.txt', 'new.txt')       # Rename file

# Using pathlib (Python 3.4+)
from pathlib import Path

path = Path('folder/file.txt')
print(path.exists())
print(path.is_file())
print(path.is_dir())
print(path.name)        # file.txt
print(path.stem)        # file
print(path.suffix)      # .txt
print(path.parent)      # folder
```

```python
# Create directory
Path('new_folder').mkdir(exist_ok=True)

# Read/write with pathlib
path = Path('file.txt')
content = path.read_text()
path.write_text('New content')
```

Binary Files

```python
# Write binary data
data = bytes([0, 1, 2, 3, 4])
with open('data.bin', 'wb') as file:
    file.write(data)

# Read binary data
with open('data.bin', 'rb') as file:
    data = file.read()
    print(data)
```

Exception Handling

Try-Except

```python
# Basic try-except
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")

# Multiple exceptions
try:
    x = int("abc")
except ValueError:
    print("Invalid value")
except TypeError:
    print("Type error")

# Catch multiple exceptions
try:
    # code
    pass
except (ValueError, TypeError):
    print("Value or Type error")

# Catch all exceptions (not recommended)
try:
```

```
    # code
    pass
except Exception as e:
    print(f"Error: {e}")
```

Try-Except-Else

```
try:
    x = int(input("Enter a number: "))
except ValueError:
    print("Invalid input")
else:
    # Executes if no exception
    print(f"You entered: {x}")
```

Try-Except-Finally

```
try:
    file = open('file.txt', 'r')
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    # Always executes
    if 'file' in locals():
        file.close()
```

Raising Exceptions

```
# Raise exception
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

# Raise with custom message
def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b

# Re-raise exception
try:
    x = 10 / 0
except ZeroDivisionError:
```

```
    print("Error occurred")
    raise  # Re-raise the same exception
```

Custom Exceptions

```
# Define custom exception
class CustomError(Exception):
    pass

class ValueTooLargeError(Exception):
    def __init__(self, value, limit):
        self.value = value
        self.limit = limit
        super().__init__(f"Value {value} exceeds limit {limit}")

# Use custom exception
def check_value(value):
    if value > 100:
        raise ValueTooLargeError(value, 100)

try:
    check_value(150)
except ValueTooLargeError as e:
    print(e)
```

Common Built-in Exceptions

```
# ValueError - Invalid value
int("abc")

# TypeError - Wrong type
"2" + 2

# KeyError - Key not found
d = {'a': 1}
d['b']

# IndexError - Index out of range
lst = [1, 2, 3]
lst[10]

# AttributeError - Attribute not found
x = 5
x.append(1)

# FileNotFoundError - File doesn't exist
open('nonexistent.txt')

# ZeroDivisionError - Division by zero
```

```
10 / 0

# ImportError - Module import fails
import nonexistent_module

# RuntimeError - General runtime error
```

Assert Statement

```
# Assert for debugging
x = 5
assert x > 0, "x must be positive"

# Assertions can be disabled with -O flag
# python -O script.py

def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"
    return sum(numbers) / len(numbers)
```

Object-Oriented Programming

Classes and Objects

```
# Define class
class Person:
    pass

# Create object
person1 = Person()

# Class with attributes
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person1 = Person("Alice", 25)
print(person1.name)  # Alice
print(person1.age)   # 25
```

Methods

```
class Person:
    def __init__(self, name, age):
```

```
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name}"

    def have_birthday(self):
        self.age += 1

person = Person("Alice", 25)
print(person.greet())
person.have_birthday()
print(person.age)  # 26
```

Class Variables vs Instance Variables

```
class Dog:
    # Class variable (shared by all instances)
    species = "Canis familiaris"

    def __init__(self, name, age):
        # Instance variables (unique to each instance)
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.species)  # Canis familiaris
print(dog2.species)  # Canis familiaris
print(dog1.name)     # Buddy
print(dog2.name)     # Max

# Modify class variable
Dog.species = "Dog"
print(dog1.species)  # Dog
print(dog2.species)  # Dog
```

Encapsulation (Private Attributes)

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
```

```
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())  # 1500
# print(account.__balance)    # AttributeError

# Name mangling (can still access with _ClassName__attribute)
print(account._BankAccount__balance)  # 1500
```

Inheritance

```
# Base class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

# Derived class
class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

dog = Dog("Buddy")
cat = Cat("Whiskers")
print(dog.speak())  # Buddy says Woof!
print(cat.speak())  # Whiskers says Meow!

# Call parent method
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent __init__
        self.breed = breed
```

Multiple Inheritance

```
class Flyer:
    def fly(self):
        return "Flying"

class Swimmer:
    def swim(self):
        return "Swimming"
```

```python
class Duck(Flyer, Swimmer):
    def quack(self):
        return "Quack"

duck = Duck()
print(duck.fly())    # Flying
print(duck.swim())   # Swimming
print(duck.quack())  # Quack
```

Method Types

Instance Methods

```python
class MyClass:
    def instance_method(self):
        return f"Instance method called by {self}"
```

Class Methods

```python
class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

person1 = Person("Alice")
person2 = Person("Bob")
print(Person.get_count())  # 2
```

Static Methods

```python
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def multiply(a, b):
        return a * b
```

```
print(MathUtils.add(5, 3))       # 8
print(MathUtils.multiply(5, 3))  # 15
```

Property Decorators

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

temp = Temperature(25)
print(temp.celsius)      # 25
print(temp.fahrenheit)   # 77.0
temp.fahrenheit = 86
print(temp.celsius)      # 30.0
```

Magic Methods (Dunder Methods)

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        # Called by str() and print()
        return f"Point({self.x}, {self.y})"

    def __repr__(self):
        # Official string representation
        return f"Point({self.x}, {self.y})"

    def __add__(self, other):
```

```
        # Overload + operator
        return Point(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        # Overload == operator
        return self.x == other.x and self.y == other.y

    def __len__(self):
        return 2

    def __getitem__(self, index):
        if index == 0:
            return self.x
        elif index == 1:
            return self.y
        raise IndexError("Index out of range")

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1)           # Point(1, 2)
p3 = p1 + p2        # Point(4, 6)
print(p1 == p2)     # False
print(len(p1))      # 2
print(p1[0])        # 1
```

Abstract Classes

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# shape = Shape()  # Error: Cannot instantiate abstract class
```

```
rect = Rectangle(5, 3)
print(rect.area())  # 15
```

Modules and Packages

Importing Modules

```
# Import entire module
import math
print(math.sqrt(16))  # 4.0

# Import specific items
from math import sqrt, pi
print(sqrt(16))  # 4.0
print(pi)        # 3.14159...

# Import with alias
import math as m
print(m.sqrt(16))

# Import all (not recommended)
from math import *

# Import from package
from collections import Counter
```

Creating Modules

my_module.py

```
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b

PI = 3.14159
```

Using the module

```
import my_module

print(my_module.greet("Alice"))
print(my_module.add(5, 3))
print(my_module.PI)
```

```
# Or
from my_module import greet, add
print(greet("Bob"))
print(add(10, 20))
```

Creating Packages

Directory structure:

```
my_package/
    __init__.py
    module1.py
    module2.py
    subpackage/
        __init__.py
        module3.py
```

init.py

```
# Can be empty or contain initialization code
from .module1 import function1
from .module2 import function2

__all__ = ['function1', 'function2']
```

Using the package

```
import my_package
from my_package import module1
from my_package.subpackage import module3
```

Module Special Variables

```
# __name__ variable
if __name__ == "__main__":
    # This code runs only when script is executed directly
    print("Running as main program")

# __file__ variable
import os
print(__file__)  # Full path to current file
print(os.path.dirname(__file__))  # Directory of current file
```

Useful Standard Library Modules

```
# os - Operating system interface
import os
os.getcwd()
os.listdir('.')
os.path.exists('file.txt')

# sys - System-specific parameters
import sys
sys.version
sys.argv  # Command line arguments
sys.exit()

# datetime - Date and time
import datetime
datetime.datetime.now()
datetime.date.today()

# random - Random number generation
import random
random.random()
random.randint(1, 10)
random.choice([1, 2, 3, 4, 5])

# json - JSON encoding/decoding
import json
json.dumps({'name': 'Alice'})
json.loads('{"name": "Alice"}')

# re - Regular expressions
import re
re.search(r'\d+', 'abc123')
re.findall(r'\d+', 'abc123xyz456')
```

List Comprehensions

Basic List Comprehension

```
# Traditional way
squares = []
for x in range(10):
    squares.append(x**2)

# List comprehension
squares = [x**2 for x in range(10)]

# With condition
```

```
evens = [x for x in range(10) if x % 2 == 0]

# Multiple conditions
numbers = [x for x in range(20) if x % 2 == 0 if x % 3 == 0]
```

Nested List Comprehension

```
# 2D list
matrix = [[i*j for j in range(3)] for i in range(3)]
# [[0, 0, 0], [0, 1, 2], [0, 2, 4]]

# Flatten 2D list
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dictionary Comprehension

```
# Create dictionary
squares = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# From two lists
keys = ['a', 'b', 'c']
values = [1, 2, 3]
dict_comp = {k: v for k, v in zip(keys, values)}

# With condition
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}

# Swap keys and values
original = {'a': 1, 'b': 2, 'c': 3}
swapped = {v: k for k, v in original.items()}
```

Set Comprehension

```
# Create set
squares = {x**2 for x in range(10)}

# With condition
even_squares = {x**2 for x in range(10) if x % 2 == 0}
```

Generator Expression

```
# Generator (uses parentheses)
gen = (x**2 for x in range(10))
print(next(gen))  # 0
print(next(gen))  # 1

# Convert to list
squares = list(x**2 for x in range(10))

# Memory efficient for large datasets
sum_of_squares = sum(x**2 for x in range(1000000))
```

Lambda Functions

Basic Lambda

```
# Regular function
def add(a, b):
    return a + b

# Lambda equivalent
add = lambda a, b: a + b
print(add(5, 3))  # 8

# Lambda with single argument
square = lambda x: x**2
print(square(5))  # 25
```

Lambda with Built-in Functions

map()

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]
```

filter()

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4, 6, 8, 10]
```

sorted()

```
students = [
    {'name': 'Alice', 'grade': 85},
    {'name': 'Bob', 'grade': 92},
    {'name': 'Charlie', 'grade': 78}
]
sorted_students = sorted(students, key=lambda x: x['grade'])
```

reduce()

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
# 120
```

Lambda Limitations

```
# Lambda can only contain expressions, not statements
# No multiple lines
# No assignments
# No annotations

# Don't do this (use regular function instead)
# complex_lambda = lambda x: (
#     temp = x * 2,  # Error: can't assign
#     temp + 1
# )
```

Decorators

Basic Decorator

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

```
# Output:
# Before function call
# Hello!
# After function call
```

Decorator with Arguments

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

@my_decorator
def greet(name):
    print(f"Hello, {name}!")
    return "Done"

result = greet("Alice")
```

Decorator with Parameters

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
# Prints "Hello, Alice!" three times
```

Preserving Function Metadata

```
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
```

```
        """Wrapper function"""
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def greet(name):
    """Greet someone"""
    print(f"Hello, {name}!")

print(greet.__name__)  # greet (not wrapper)
print(greet.__doc__)   # Greet someone
```

Class Decorators

```
class CountCalls:
    def __init__(self, func):
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"Call {self.count} of {self.func.__name__}")
        return self.func(*args, **kwargs)

@CountCalls
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
greet("Bob")
```

Common Built-in Decorators

```
class MyClass:
    @staticmethod
    def static_method():
        print("Static method")

    @classmethod
    def class_method(cls):
        print(f"Class method of {cls}")

    @property
    def my_property(self):
        return "Property value"
```

Practical Decorator Examples

Timer Decorator

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.2f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(2)

slow_function()
```

Memoization Decorator

```
def memoize(func):
    cache = {}
    @wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@memoize
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Generators and Iterators

Iterators

```
# Create iterator from list
numbers = [1, 2, 3, 4, 5]
iterator = iter(numbers)
```

```
print(next(iterator))  # 1
print(next(iterator))  # 2
print(next(iterator))  # 3

# Iterate using for loop
for num in numbers:
    print(num)
```

Custom Iterator

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1

counter = Counter(1, 5)
for num in counter:
    print(num)  # 1, 2, 3, 4, 5
```

Generators (using yield)

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for num in countdown(5):
    print(num)  # 5, 4, 3, 2, 1

# Generator stores state
gen = countdown(3)
print(next(gen))  # 3
print(next(gen))  # 2
print(next(gen))  # 1
```

Generator Expressions

```python
# Generator expression (memory efficient)
gen = (x**2 for x in range(1000000))
print(next(gen))  # 0
print(next(gen))  # 1

# List comprehension (loads all into memory)
lst = [x**2 for x in range(1000000)]
```

Generator Examples

Fibonacci Generator

```python
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib))
```

Reading Large Files

```python
def read_large_file(file_path):
    with open(file_path, 'r') as file:
        for line in file:
            yield line.strip()

for line in read_large_file('large_file.txt'):
    process(line)
```

yield from

```python
def generator1():
    yield 1
    yield 2

def generator2():
    yield from generator1()
    yield 3
    yield 4

for num in generator2():
    print(num)  # 1, 2, 3, 4
```

Regular Expressions

Basic Pattern Matching

```
import re

# Search for pattern
text = "The price is 100 dollars"
match = re.search(r'\d+', text)
if match:
    print(match.group())  # 100

# Find all matches
text = "Phone: 123-456-7890, Mobile: 098-765-4321"
matches = re.findall(r'\d{3}-\d{3}-\d{4}', text)
print(matches)  # ['123-456-7890', '098-765-4321']

# Match at beginning
text = "Python is awesome"
if re.match(r'Python', text):
    print("Starts with Python")
```

Pattern Compilation

```
# Compile pattern for reuse
pattern = re.compile(r'\d+')
matches = pattern.findall("123 abc 456 def 789")
print(matches)  # ['123', '456', '789']
```

Common Patterns

```
# Digits
re.findall(r'\d', 'abc123def456')      # ['1', '2', '3', '4', '5', '6']
re.findall(r'\d+', 'abc123def456')     # ['123', '456']

# Word characters
re.findall(r'\w+', 'hello world!')     # ['hello', 'world']

# Whitespace
re.findall(r'\s+', 'hello   world')    # ['   ']

# Any character except newline
re.findall(r'.', 'abc')                # ['a', 'b', 'c']

# Beginning and end
```

```
re.match(r'^Hello', 'Hello World')     # Match
re.search(r'World$', 'Hello World')    # Match
```

Character Classes

```
# Character class
re.findall(r'[aeiou]', 'hello')        # ['e', 'o']

# Range
re.findall(r'[a-z]', 'Hello123')       # ['e', 'l', 'l', 'o']
re.findall(r'[A-Z]', 'Hello123')       # ['H']

# Negation
re.findall(r'[^0-9]', 'abc123')        # ['a', 'b', 'c']
```

Quantifiers

```
# * (0 or more)
re.findall(r'ab*', 'a ab abb abbb')    # ['a', 'ab', 'abb', 'abbb']

# + (1 or more)
re.findall(r'ab+', 'a ab abb abbb')    # ['ab', 'abb', 'abbb']

# ? (0 or 1)
re.findall(r'ab?', 'a ab abb')         # ['a', 'ab', 'ab']

# {n} (exactly n)
re.findall(r'a{3}', 'aa aaa aaaa')     # ['aaa', 'aaa']

# {n,} (n or more)
re.findall(r'a{2,}', 'a aa aaa')       # ['aa', 'aaa']

# {n,m} (between n and m)
re.findall(r'a{2,3}', 'a aa aaa aaaa') # ['aa', 'aaa', 'aaa']
```

Groups and Capturing

```
# Capturing groups
text = "John Doe, email: john@example.com"
match = re.search(r'(\w+)\s+(\w+).*?(\S+@\S+)', text)
if match:
    print(match.group(0))  # Full match
    print(match.group(1))  # John
    print(match.group(2))  # Doe
    print(match.group(3))  # john@example.com
```

```
# Named groups
match = re.search(r'(?P<first>\w+)\s+(?P<last>\w+)', text)
if match:
    print(match.group('first'))  # John
    print(match.group('last'))   # Doe
```

Substitution

```
# Replace pattern
text = "The price is 100 dollars"
result = re.sub(r'\d+', 'XXX', text)
print(result)  # The price is XXX dollars

# Replace with function
def increment(match):
    return str(int(match.group()) + 1)

result = re.sub(r'\d+', increment, text)
print(result)  # The price is 101 dollars

# Backreferences
text = "John Doe"
result = re.sub(r'(\w+)\s+(\w+)', r'\2, \1', text)
print(result)  # Doe, John
```

Split

```
# Split by pattern
text = "apple,banana;cherry:date"
parts = re.split(r'[,;:]', text)
print(parts)  # ['apple', 'banana', 'cherry', 'date']
```

Flags

```
# Case insensitive
re.findall(r'python', 'Python PYTHON python', re.IGNORECASE)

# Multiline
text = "Line 1\nLine 2\nLine 3"
re.findall(r'^Line', text, re.MULTILINE)

# Dot matches newline
re.search(r'.*', 'Line 1\nLine 2', re.DOTALL)

# Verbose (allows comments)
pattern = re.compile(r'''
```

```
    \d{3}  # Area code
    -      # Separator
    \d{3}  # First three digits
    -      # Separator
    \d{4}  # Last four digits
''', re.VERBOSE)
```

Working with Dates and Times

datetime Module

```
from datetime import datetime, date, time, timedelta

# Current date and time
now = datetime.now()
print(now)

# Current date
today = date.today()
print(today)

# Create specific date
dt = datetime(2024, 1, 15, 14, 30, 0)
print(dt)

# Create specific date
d = date(2024, 1, 15)
print(d)

# Create specific time
t = time(14, 30, 0)
print(t)
```

Formatting Dates

```
from datetime import datetime

now = datetime.now()

# strftime (datetime to string)
print(now.strftime("%Y-%m-%d"))           # 2024-01-15
print(now.strftime("%B %d, %Y"))          # January 15, 2024
print(now.strftime("%I:%M %p"))           # 02:30 PM
print(now.strftime("%A, %B %d, %Y"))      # Monday, January 15, 2024

# Common format codes:
# %Y - Year (4 digits)
# %m - Month (01-12)
```

```
# %d - Day (01-31)
# %H - Hour (00-23)
# %I - Hour (01-12)
# %M - Minute (00-59)
# %S - Second (00-59)
# %p - AM/PM
# %A - Weekday name
# %B - Month name
```

Parsing Dates

```
from datetime import datetime

# strptime (string to datetime)
date_string = "2024-01-15"
dt = datetime.strptime(date_string, "%Y-%m-%d")
print(dt)

date_string = "January 15, 2024"
dt = datetime.strptime(date_string, "%B %d, %Y")
print(dt)
```

Date Arithmetic

```
from datetime import datetime, timedelta

now = datetime.now()

# Add days
future = now + timedelta(days=7)
print(future)

# Subtract days
past = now - timedelta(days=7)
print(past)

# Timedelta with multiple units
delta = timedelta(days=7, hours=3, minutes=30)
future = now + delta

# Difference between dates
date1 = datetime(2024, 1, 15)
date2 = datetime(2024, 1, 1)
difference = date1 - date2
print(difference.days)  # 14
```

Date Components

```python
from datetime import datetime

dt = datetime.now()
print(dt.year)        # 2024
print(dt.month)       # 1
print(dt.day)         # 15
print(dt.hour)        # 14
print(dt.minute)      # 30
print(dt.second)      # 0
print(dt.weekday())   # 0 (Monday=0, Sunday=6)
```

Time Zones

```python
from datetime import datetime, timezone
import pytz

# UTC time
utc_now = datetime.now(timezone.utc)
print(utc_now)

# Specific timezone (requires pytz)
eastern = pytz.timezone('US/Eastern')
eastern_time = datetime.now(eastern)
print(eastern_time)

# Convert timezone
utc_time = datetime.now(timezone.utc)
eastern_time = utc_time.astimezone(eastern)
```

JSON and CSV

JSON

Reading JSON

```python
import json

# From string
json_string = '{"name": "Alice", "age": 25, "city": "New York"}'
data = json.loads(json_string)
print(data['name'])  # Alice

# From file
with open('data.json', 'r') as file:
    data = json.load(file)
    print(data)
```

Writing JSON

```
import json

# To string
data = {"name": "Alice", "age": 25, "city": "New York"}
json_string = json.dumps(data)
print(json_string)

# To string (formatted)
json_string = json.dumps(data, indent=4)
print(json_string)

# To file
with open('data.json', 'w') as file:
    json.dump(data, file, indent=4)
```

Working with JSON

```
import json

# Complex data
data = {
    "name": "Alice",
    "age": 25,
    "hobbies": ["reading", "gaming", "coding"],
    "address": {
        "street": "123 Main St",
        "city": "New York"
    }
}

json_string = json.dumps(data, indent=2)
print(json_string)

# Parse back
parsed = json.loads(json_string)
print(parsed['hobbies'][0])  # reading
```

CSV

Reading CSV

```
import csv

# Read as list of lists
```

```python
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Read as dictionary
with open('data.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row['name'], row['age'])

# With specific delimiter
with open('data.tsv', 'r') as file:
    reader = csv.reader(file, delimiter='\t')
    for row in reader:
        print(row)
```

Writing CSV

```python
import csv

# Write list of lists
data = [
    ['Name', 'Age', 'City'],
    ['Alice', 25, 'New York'],
    ['Bob', 30, 'London']
]

with open('output.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(data)

# Write dictionary
data = [
    {'name': 'Alice', 'age': 25, 'city': 'New York'},
    {'name': 'Bob', 'age': 30, 'city': 'London'}
]

with open('output.csv', 'w', newline='') as file:
    fieldnames = ['name', 'age', 'city']
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

Virtual Environments

Creating Virtual Environment

```
# Create virtual environment
python -m venv myenv

# Or using virtualenv
virtualenv myenv
```

Activating Virtual Environment

```
# Windows
myenv\Scripts\activate

# macOS/Linux
source myenv/bin/activate
```

Deactivating Virtual Environment

```
deactivate
```

Installing Packages

```
# Install package
pip install requests

# Install specific version
pip install requests==2.28.0

# Install from requirements.txt
pip install -r requirements.txt

# Uninstall package
pip uninstall requests
```

Managing Dependencies

```
# Generate requirements.txt
pip freeze > requirements.txt

# List installed packages
pip list

# Show package info
pip show requests
```

Built-in Functions

Type Conversion

```
int("10")           # 10
float("3.14")       # 3.14
str(100)            # "100"
bool(1)             # True
list("abc")         # ['a', 'b', 'c']
tuple([1, 2, 3])    # (1, 2, 3)
set([1, 2, 2, 3])   # {1, 2, 3}
dict([('a', 1)])    # {'a': 1}
```

Math Functions

```
abs(-5)             # 5
pow(2, 3)           # 8
round(3.7)          # 4
round(3.14159, 2)   # 3.14
max(1, 2, 3)        # 3
min(1, 2, 3)        # 1
sum([1, 2, 3])      # 6
```

Sequence Functions

```
len([1, 2, 3])           # 3
sorted([3, 1, 2])        # [1, 2, 3]
reversed([1, 2, 3])      # <reversed object>
enumerate(['a', 'b'])    # <enumerate object>
zip([1, 2], ['a', 'b']) # <zip object>
all([True, True])        # True
any([False, True])       # True
```

Input/Output

```
print("Hello")
name = input("Enter name: ")
```

Object Functions

```
type(5)                 # <class 'int'>
isinstance(5, int)      # True
```

```
id(x)                   # Object id
dir(object)             # List attributes
help(function)          # Help documentation
vars(object)            # __dict__ attribute
```

Other Useful Functions

```
range(5)                # 0, 1, 2, 3, 4
map(func, iterable)
filter(func, iterable)
chr(65)                 # 'A'
ord('A')                # 65
eval("2 + 2")           # 4 (use carefully!)
exec("x = 5")           # Execute code (use carefully!)
```

Common Standard Library Modules

collections

```
from collections import Counter, defaultdict, OrderedDict, deque

# Counter
counter = Counter(['a', 'b', 'a', 'c', 'b', 'a'])
print(counter.most_common(2))  # [('a', 3), ('b', 2)]

# defaultdict
dd = defaultdict(list)
dd['key'].append(1)

# deque (double-ended queue)
dq = deque([1, 2, 3])
dq.append(4)
dq.appendleft(0)
dq.pop()
dq.popleft()
```

itertools

```
from itertools import count, cycle, repeat, chain, combinations,
permutations

# Infinite iterators
# count(10) → 10, 11, 12, 13, ...
# cycle('ABC') → A, B, C, A, B, C, ...
# repeat(10, 3) → 10, 10, 10
```

```
# Combinatoric
list(combinations([1, 2, 3], 2))    # [(1, 2), (1, 3), (2, 3)]
list(permutations([1, 2, 3], 2))    # [(1, 2), (1, 3), (2, 1), ...]

# Chain
list(chain([1, 2], [3, 4]))         # [1, 2, 3, 4]
```

math

```
import math

math.sqrt(16)      # 4.0
math.ceil(3.2)     # 4
math.floor(3.8)    # 3
math.pi            # 3.14159...
math.e             # 2.71828...
math.factorial(5)  # 120
```

random

```
import random

random.random()              # Random float [0.0, 1.0)
random.randint(1, 10)        # Random int [1, 10]
random.choice([1, 2, 3])     # Random element
random.shuffle(list)         # Shuffle in place
random.sample([1,2,3,4], 2)  # Random sample
```

pathlib

```
from pathlib import Path

# Path operations
path = Path('folder/file.txt')
path.exists()
path.is_file()
path.is_dir()
path.name           # file.txt
path.stem           # file
path.suffix         # .txt
path.parent         # folder

# Create directory
Path('new_folder').mkdir(exist_ok=True)
```

```
# Iterate files
for file in Path('.').glob('*.py'):
    print(file)
```

Best Practices

PEP 8 Style Guide

Naming Conventions

```
# Variables and functions: lowercase with underscores
user_name = "Alice"
def calculate_total():
    pass

# Constants: UPPERCASE with underscores
MAX_SIZE = 100
DEFAULT_COLOR = "blue"

# Classes: PascalCase
class MyClass:
    pass

# Private: prefix with underscore
_internal_value = 10

# Protected (convention): single underscore
class MyClass:
    def __init__(self):
        self._protected = 10
```

Indentation and Spacing

```
# Use 4 spaces per indentation level
def function():
    if True:
        print("Indented")

# Blank lines
# - 2 blank lines before top-level functions/classes
# - 1 blank line between methods

# Spaces around operators
x = 1 + 2
result = x * (y + 3)

# No spaces inside brackets
```

```
my_list = [1, 2, 3]
my_dict = {'key': 'value'}
```

Line Length

```
# Maximum line length: 79 characters

# Line continuation
total = (first_variable +
         second_variable +
         third_variable)

# Or
total = (first_variable + second_variable +
         third_variable)
```

Code Organization

```
# Import order:
# 1. Standard library
# 2. Third-party packages
# 3. Local modules

import os
import sys

import numpy as np
import pandas as pd

from my_module import my_function
```

Documentation

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.

    Raises:
        ValueError: If length or width is negative.
```

```
    Example:
        >>> calculate_area(5, 3)
        15
    """
    if length < 0 or width < 0:
        raise ValueError("Dimensions cannot be negative")
    return length * width
```

Common Patterns

Context Managers

```
# Always use context managers for resources
with open('file.txt', 'r') as file:
    content = file.read()

# Custom context manager
from contextlib import contextmanager

@contextmanager
def my_context():
    # Setup
    print("Entering")
    yield
    # Teardown
    print("Exiting")

with my_context():
    print("Inside")
```

EAFP vs LBYL

```
# EAFP (Easier to Ask Forgiveness than Permission)
try:
    value = my_dict['key']
except KeyError:
    value = None

# LBYL (Look Before You Leap)
if 'key' in my_dict:
    value = my_dict['key']
else:
    value = None

# Python prefers EAFP
```

Avoid Mutable Default Arguments

```
# Bad
def append_to_list(item, lst=[]):
    lst.append(item)
    return lst

# Good
def append_to_list(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

Use enumerate() instead of range(len())

```
# Bad
for i in range(len(items)):
    print(i, items[i])

# Good
for i, item in enumerate(items):
    print(i, item)
```

Use zip() for parallel iteration

```
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]

# Bad
for i in range(len(names)):
    print(f"{names[i]} is {ages[i]} years old")

# Good
for name, age in zip(names, ages):
    print(f"{name} is {age} years old")
```

Performance Tips

```
# Use list comprehension instead of loops
# Bad
squares = []
for x in range(10):
    squares.append(x**2)
```

```python
# Good
squares = [x**2 for x in range(10)]

# Use generators for large datasets
# Bad (loads all into memory)
sum([x**2 for x in range(1000000)])

# Good (memory efficient)
sum(x**2 for x in range(1000000))

# Use sets for membership testing
# Bad (O(n))
if item in my_list:
    pass

# Good (O(1))
if item in my_set:
    pass
```

Quick Reference

Data Types Summary

```python
# Numeric
int, float, complex

# Sequence
str, list, tuple, range

# Mapping
dict

# Set
set, frozenset

# Boolean
bool

# Binary
bytes, bytearray, memoryview

# None
NoneType
```

Common Operations

```python
# String
s.upper(), s.lower(), s.strip(), s.split(), s.replace()
```

```
s.startswith(), s.endswith(), s.find(), s.count()

# List
lst.append(), lst.insert(), lst.remove(), lst.pop()
lst.sort(), lst.reverse(), lst.clear()

# Dictionary
d.keys(), d.values(), d.items(), d.get()
d.pop(), d.update(), d.clear()

# Set
s.add(), s.remove(), s.discard()
s.union(), s.intersection(), s.difference()
```

File Operations

```
# Read
with open('file.txt', 'r') as f:
    content = f.read()

# Write
with open('file.txt', 'w') as f:
    f.write('content')

# Append
with open('file.txt', 'a') as f:
    f.write('more content')
```